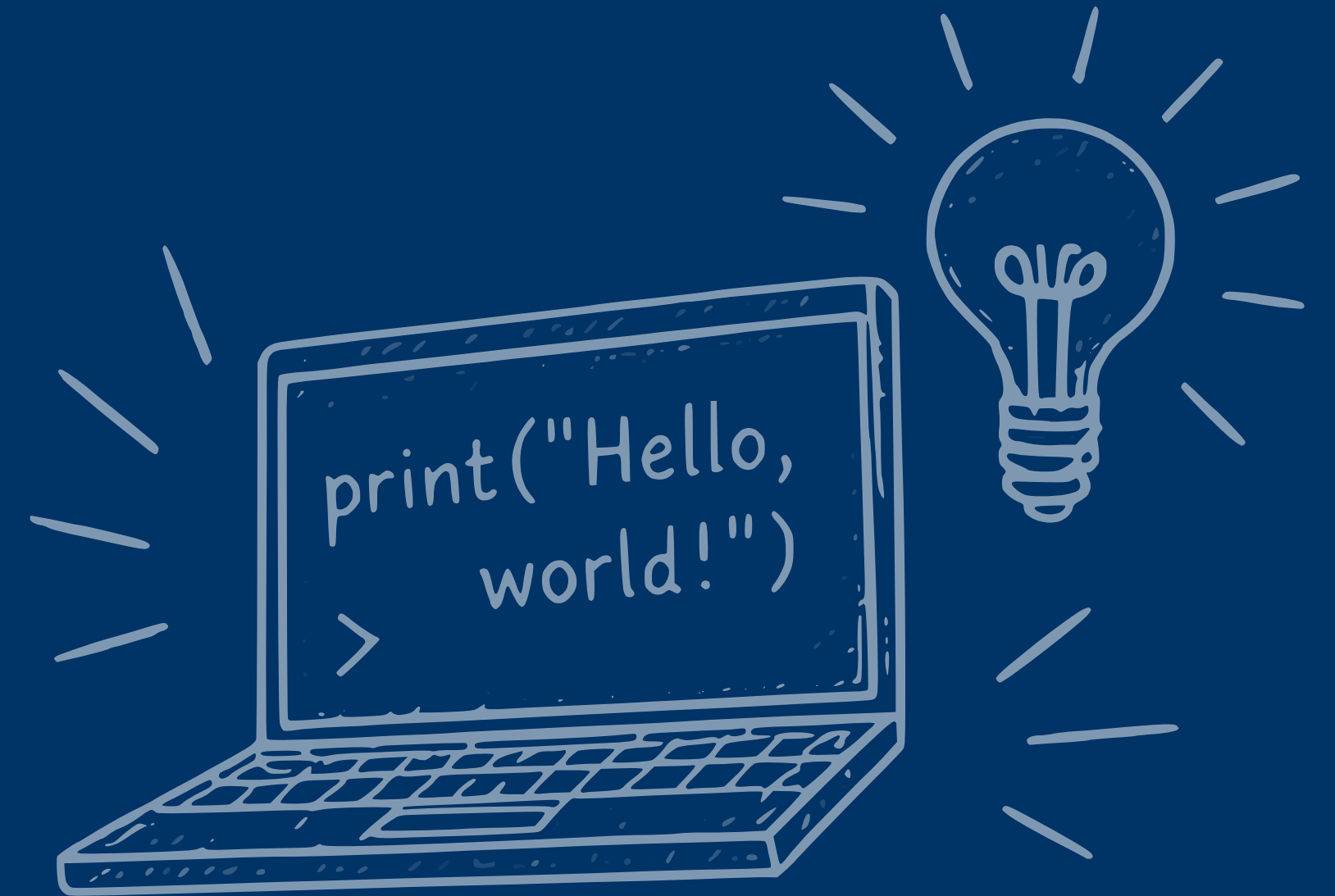


Programming

Lecture 01 — Foundations

Computers, algorithms, and your first program



Dr. Ramon Winterhalder

Human-Centered Artificial Intelligence · A.Y. 2026/27



UNIVERSITÀ
DEGLI STUDI
DI MILANO



Practical matters

Two minutes, then we start

Where everything lives

- **GitHub** — slides, notebooks, exercises and solutions.
- **myAriel** — please enrol: announcements, exam dates.

12 lectures × 4 hours

- 11 teaching lectures
- plus 1 final recap with a full mock exam.

Laptops from lecture 02

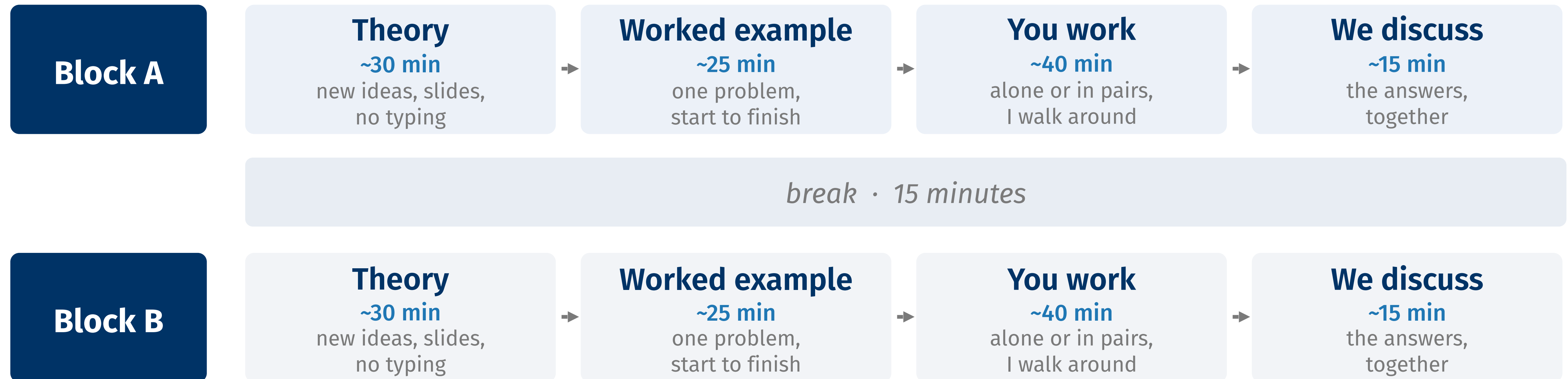
- Today you can work entirely on paper.
- From next week, bring one if you can.

<https://github.com/ramonpeter/programming-hcai-2026>

- Assessment: one written exam — concepts, reading code, and writing code.
- Exercise sheets are not graded and not handed in. Do them anyway!!
- Ask questions during the lecture. Interrupting me is the correct behaviour.

How a lecture works

Every week, the same shape



Four hours, two halves, four steps each. Two topics a week — they are always related, and the second usually needs the first.

You are meant to get stuck

The 40 minutes exist for that.
Ten minutes stuck is productive;
an hour is not — ask.

Slides and notebook go up after

So do not copy from the screen. Listen,
then read it later.

Sheets at the end, solutions after

You get the exercise sheet when we start
on it, and the answers once we have
discussed them.

Where we are going

The 12 lectures

01 Foundations

02 Data & expressions

03 Input/output & conditionals

04 Loops

05 Strings

06 Lists

07 Dictionaries, tuples, sets

08 Functions

09 Modules & files

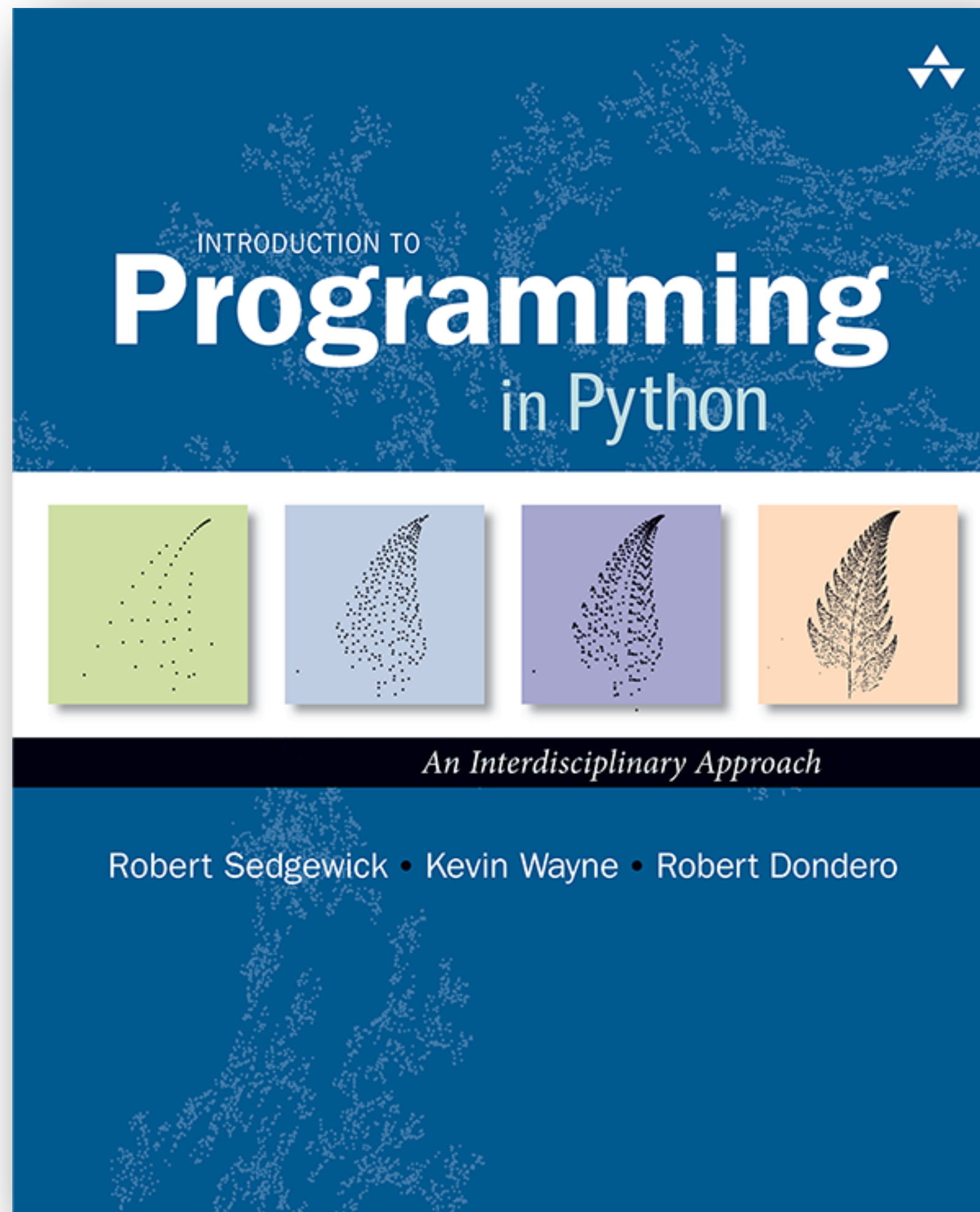
10 Recursion

11 Object-oriented programming

12 Recap & mock exam

Everything builds on what came before. Missing a lecture costs you.

Book — Introduction to programming in Python





INTRODUCTION TO PROGRAMMING IN PYTHON

*a textbook for a first course in computer science
for the next generation
of scientists and engineers*

Textbook

Our textbook *Introduction to Programming in Python* [[Amazon](#) • [Pearson](#)] is an interdisciplinary approach to the traditional CS1 curriculum. We teach all of the classic elements of programming, using an "objects-in-the-middle" approach that emphasizes data abstraction. A key feature of the book is the manner in which we motivate each programming concept by examining its impact on specific applications, taken from fields ranging from materials science to genomics to astrophysics to internet commerce. The book is organized around four stages of learning to program:

- *Chapter 1: Elements of Programming* introduces variables; assignment statements; built-in types of data; conditionals and loops; arrays; and input/output, including graphics and sound.
- *Chapter 2: Functions and Modules* introduces modular programming. We stress the fundamental idea of dividing a program into components that can be

<https://introcs.cs.princeton.edu/python/home/>

Online — The classic way

Python 3.14.7 documentation

Welcome! This is the official documentation for Python 3.14.7.

Documentation sections:

What's new in Python 3.14?

Or all "What's new" documents since Python 2.0

Tutorial

Start here: a tour of Python's syntax and features

Built-ins reference

Built-in functions and classes

Library reference

Standard library modules

Language reference

Syntax and language elements

Python setup and usage

How to install, configure, and use Python

Python HOWTOs

In-depth topic manuals

Installing Python modules

Third-party modules and PyPI.org

Extending and embedding

For C/C++ programmers

Python's C API

C API reference

FAQs

Frequently asked questions (with answers!)

Deprecations

Deprecated functionality

Other resources:

Python developer's guide

Information on contributing to Python

Python Packaging User Guide

Resources relating to Python packaging

Audio/visual talks

Podcasts, talks, and video presentations from the community

Python Enhancement Proposals

Index of proposed improvements to Python

Static Typing with Python

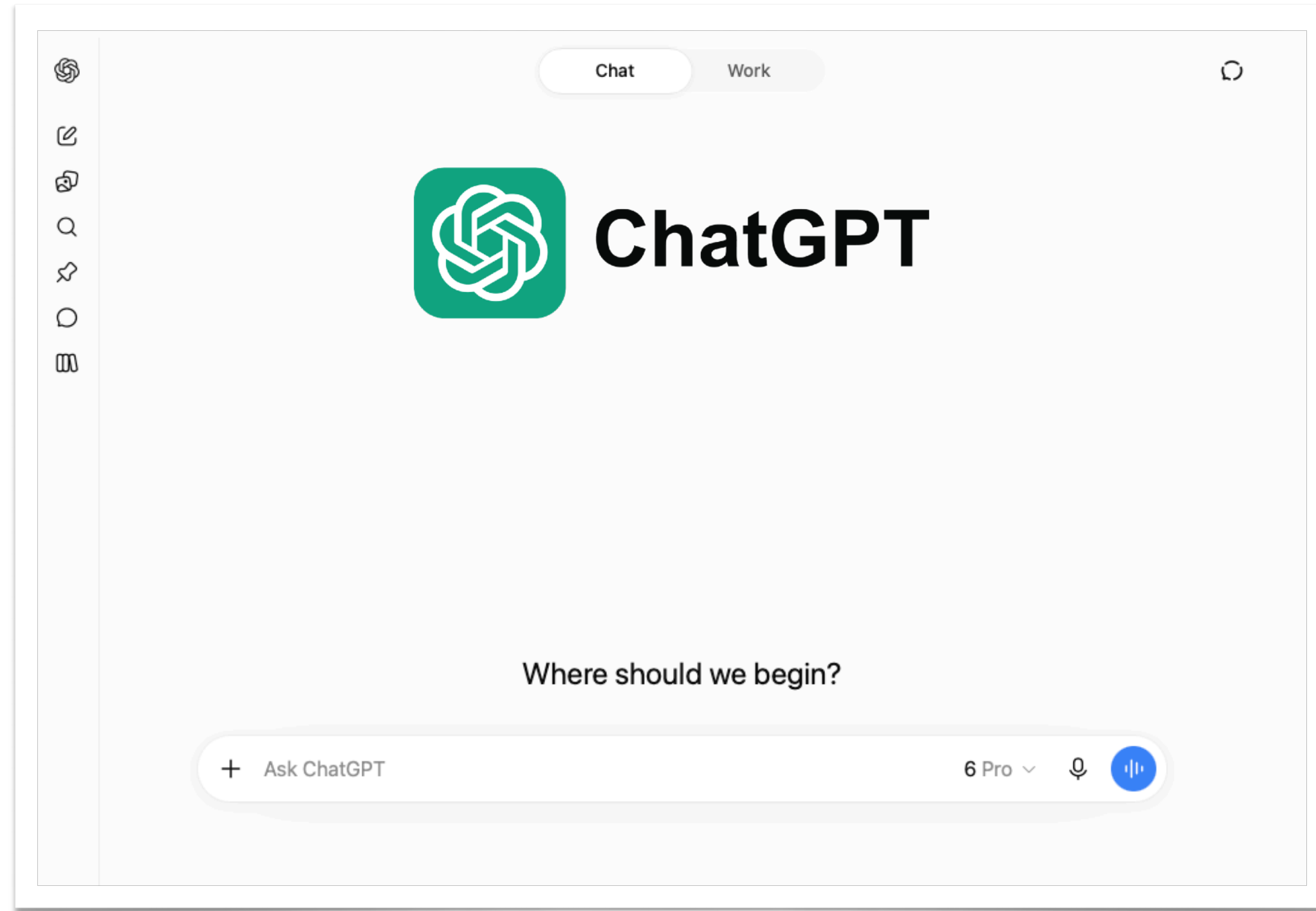
Information and guides about Python type safety

The screenshot shows the Stack Overflow homepage. At the top, there's a navigation bar with links for 'About', 'Products', 'Stack Internal', and a search bar. Below this, the 'Newest Questions' section is prominent, displaying a list of questions with their respective votes, answers, and views. The first question is 'Playwright database isolation standard' with -1 votes and 11 views. The second is 'How do I link software to "affected" section of CVE?' with -1 votes and 12 views. Below the questions, there's a sponsored section for 'Stack Overflow' with the text 'Help Build the Next Public Knowledge Exchange'. At the bottom of the main content area, there's another question 'T-SQL update error with subquery throws error' with 0 votes and 21 views. The left sidebar contains navigation links for 'Home', 'Questions', 'AI Assist', 'Tags', 'Stack Overflow for Agents', 'Challenges', 'Chat', 'Articles', 'Users', 'Companies', and 'Developer Story'. The right sidebar features a section for 'Try Stack Overflow for Agents' with a link to 'Explore Stack Overflow for Agents', and a 'The Overflow Blog' section with a link to 'Free (votes) for all (that want them)'.

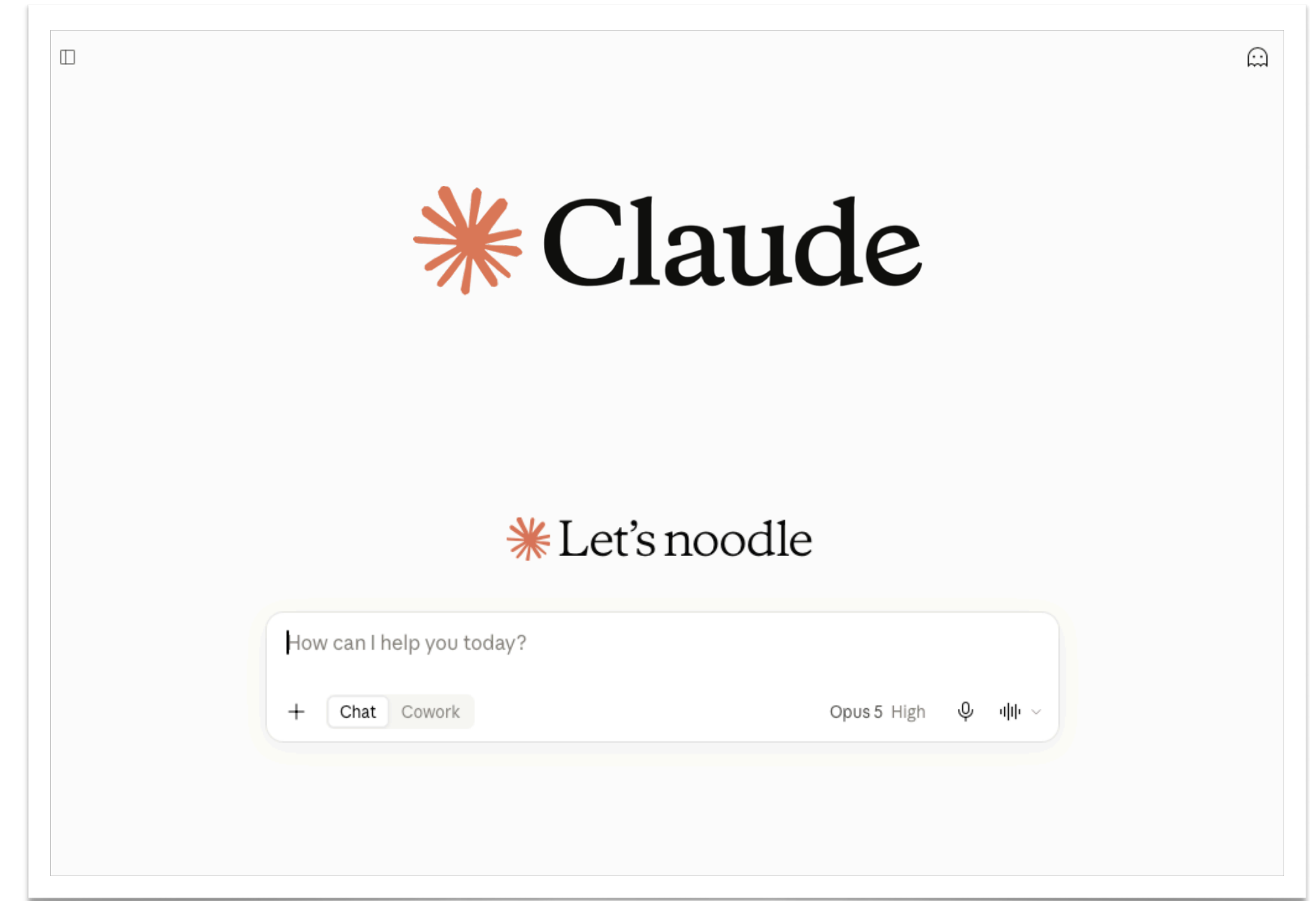
<https://docs.python.org>

<https://stackoverflow.com/>

Online — The modern way



<https://chatgpt.com>



<https://claude.ai/>

Notice: Using an AI assistant is allowed and will be useful. Using it before you have understood it yourself will make you fail the exam!



A

What is a program?

Computers · information · algorithms — no code yet

By the end of this block you can...

- explain, in one sentence, what a computer actually does;
- say why everything inside one is ultimately a number;
- define what an algorithm is — and what it is not;
- write an unambiguous algorithm for a small problem, in plain language;
- trace that algorithm by hand and find the case where it breaks.

Notice: none of these mention Python. Programming is a way of thinking; the language is a detail.

What does a computer actually do?

Less than you think

Store

Put a value somewhere and remember where

Fetch

Get that value back

Compute

Add, subtract, multiply, divide

Compare

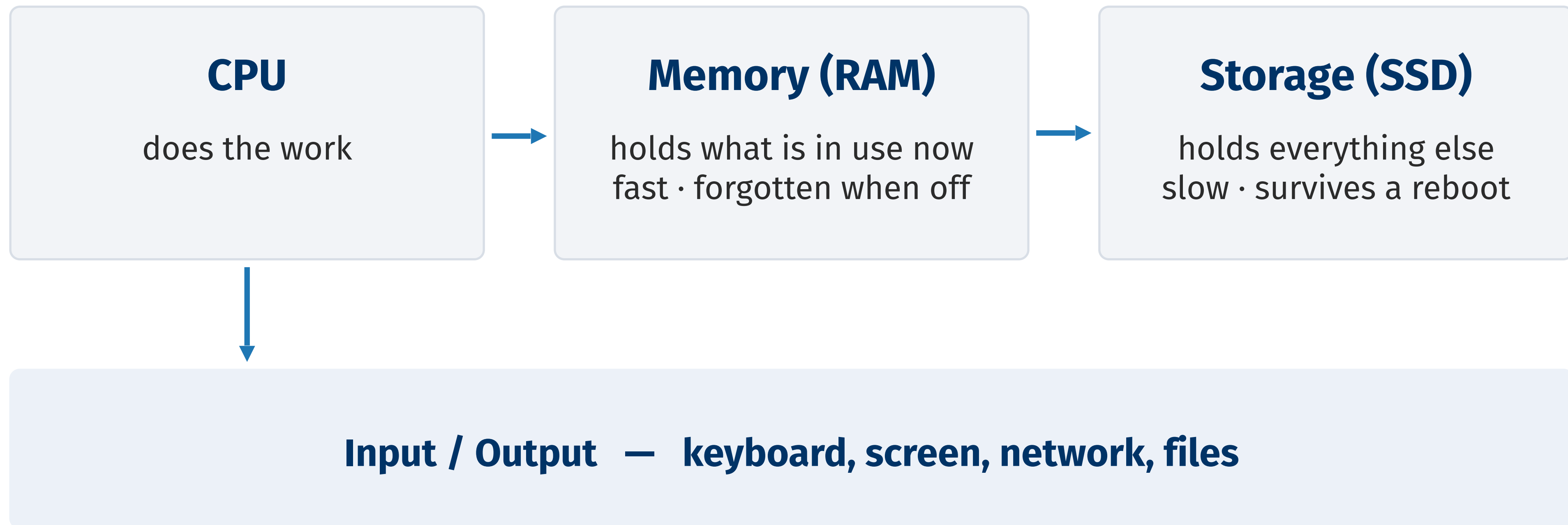
Is this bigger than that? Are they equal?

Repeat

Do the above again — billions of times per second

That is the whole repertoire. Everything else — a browser, a chatbot, a game — is these five, combined.

Anatomy of a computer



A program is a list of instructions that sits in memory and tells the CPU what to do next.

Everything is bits

A bit is a single yes/no — the only thing hardware can store

0

1

1 bit → 2 values

2 bits → 4 values

3 bits → 8 values

8 bits → 256 values (= 1 byte)

Counting in binary

binary	decimal
000	0
001	1
010	2
011	3
100	4
101	5

Same trick as decimal — each position is worth twice the one to its right, instead of ten times.

The same bits mean different things

Interpretation is everything

01000001

as a number

65

as text (ASCII)

'A'

as a pixel

a dark grey

as an instruction

something the CPU executes

The bits carry no meaning on their own. The meaning comes from the type we say they have — this is the idea behind data types, next lecture.

Levels of language

The same instruction, three ways of saying it

High-level (Python)

`x = 97`

one line per idea

Assembly

`mov al, 97`

one line per machine instruction

Machine code

`10110000 01100001`

what the CPU really executes

Going up: easier for humans, further from the hardware. We will live entirely at the top.

Why Python?

Readable

Close enough to English that you can guess what code does before you learn the rules.

Batteries included

A huge standard library, and a package for almost everything else.

The language of AI

numpy, pandas, scikit-learn, PyTorch. Every tool in your degree speaks Python.

Forgiving

No types to declare, no compilation step, immediate feedback.



The honest caveat: Python is slow for heavy numerical work. The fix is to call fast libraries from it — which is exactly what the AI stack does.

Where the name comes from

A two-minute detour — it explains something you will see all semester



Guido van Rossum
created Python

December 1989, Amsterdam

Guido van Rossum started Python over the Christmas holidays, as a side project at the national research institute CWI. First public release: February 1991.

Named after Monty Python

Not the snake. He was reading the scripts of Monty Python's Flying Circus and wanted a name that was short and linked to something cheeky.

This is why the placeholders are spam and eggs

Where other languages write foo and bar, the Python documentation writes spam, eggs and ham. You will meet them constantly — now you know why.



in most languages
foo, bar, baz

in the Python docs
spam, eggs, ham

Try it later: typing “import this” into Python prints the Zen of Python — nineteen lines on what its designers were aiming for. It has been hiding in the language since 2004.

How your code becomes action

Compiled vs interpreted

Compiled (C, C++, Fortran)



Translated once, ahead of time. Fast to run, slow to change.

Interpreted (Python)



Translated line by line, as it runs. Slower, but you see the result immediately.

Consequence for you: Python tells you about an error only when it reaches that line. A program can run for ten seconds and then crash on line 40 — line 41 was never checked.

Algorithm

A finite sequence of unambiguous steps that turns an input into the intended output.

Finite

It stops. Eventually, and for every allowed input.

Unambiguous

Each step has exactly one meaning. No judgement required.

Effective

Each step is something the executor can actually do.

Defined I/O

It is clear what goes in and what comes out.

An algorithm is independent of any programming language. A program is one algorithm, written down in one language.

Worked example 1

Write an algorithm: make a cup of tea

First attempt

1. Boil some water
2. Put a teabag in a cup
3. Add the water
4. Wait a bit
5. Add milk if you like it

Good enough for a human. Hopeless for a machine.

Where it breaks

- "some" — how much?
- "a bit" — 30 seconds or 30 minutes?
- "if you like it" — who decides, and how?
- What if there is no cup?
- What if the water never boils — when do we give up?

Worked example 1

Second attempt — unambiguous

INPUT: a cup, a teabag, 250 ml water, a kettle

1. Pour 250 ml of water into the kettle
2. Switch the kettle on
3. WHILE the water temperature is below 100 °C:
 wait 1 second
4. Place 1 teabag in the cup
5. Pour all the water from the kettle into the cup
6. Wait 180 seconds
7. Remove the teabag

OUTPUT: a cup of tea

- Every quantity is a number.
- Every decision is a test with a yes/no answer.
- "Wait a bit" became a loop with a condition.
- Inputs and outputs are stated.

Steps 3 and 6 are already a loop and a fixed repetition — the control structures of lecture 04.

Pseudocode

Our notation for today, and for the back of an envelope when you are stuck

```
SET x TO 5
```

give a name to a value

```
READ n
```

get an input

```
PRINT x
```

produce an output

```
IF condition THEN ... ELSE ...
```

choose between two paths

```
WHILE condition: ...
```

repeat while something holds

```
REPEAT n TIMES: ...
```

repeat a fixed number of times

Pseudocode has no official standard. It only has to be unambiguous to a human reader.

Worked example 2

The largest of three numbers

```
READ a, b, c

SET largest TO a

IF b > largest THEN
    SET largest TO b

IF c > largest THEN
    SET largest TO c

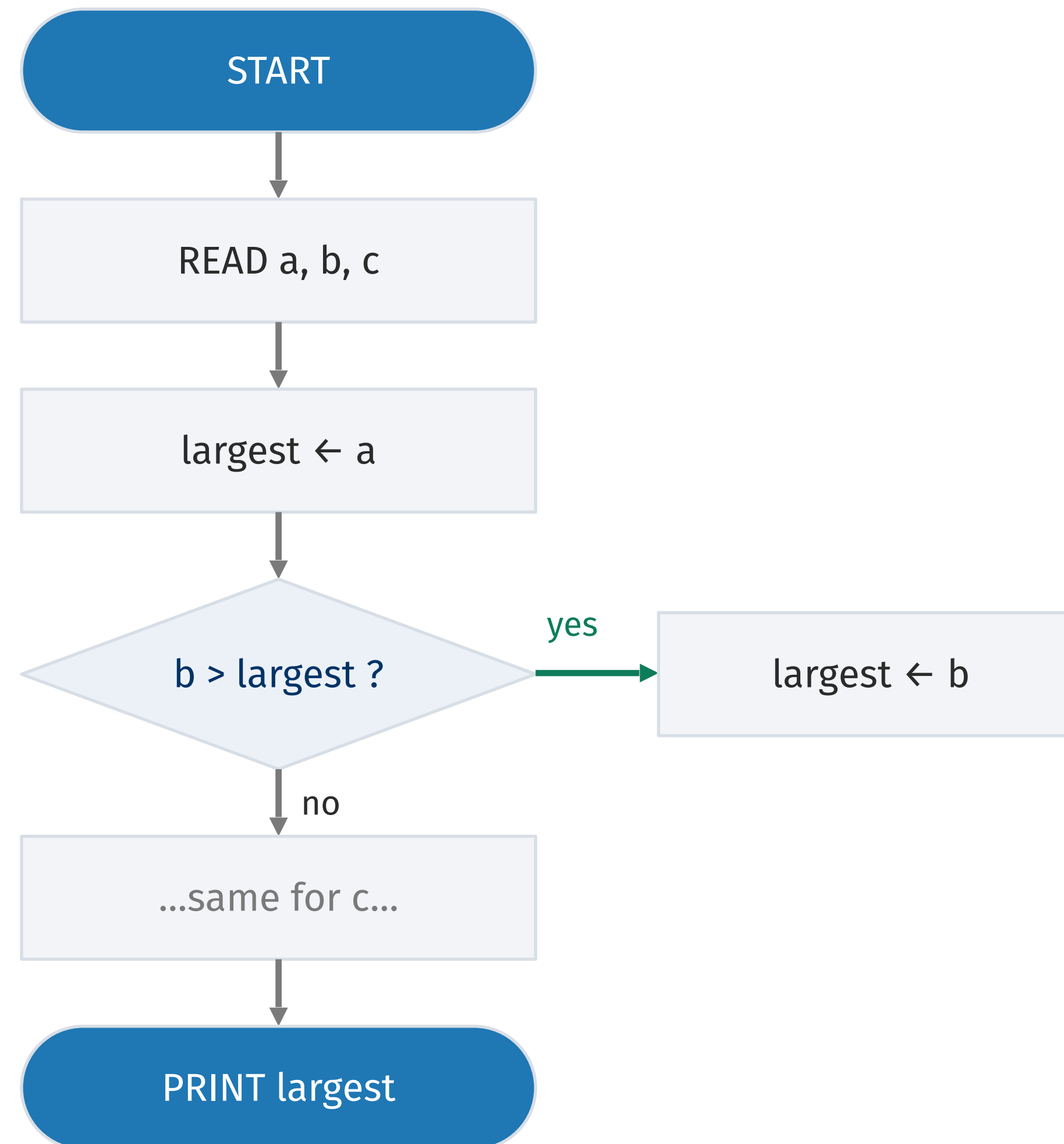
PRINT largest
```

- Assume one is the winner.
- Challenge it with each of the others.
- Whatever survives is the answer.
- Three numbers or three thousand — the shape of the idea does not change.

This pattern has a name: the running maximum.
You will see a similar pattern in other lectures again

The same algorithm as a flowchart

Boxes = do something
Diamonds = ask a yes/no question
Arrows = what happens next



Flowcharts are useful for about twenty lines and useless beyond that. We will use them today and then mostly stop.

Trace it by hand

$a = 3, b = 9, c = 5$

step	a	b	c	largest	what happened
READ a, b, c	3	9	5	—	inputs arrive
largest $\leftarrow$ a	3	9	5	3	assume a wins
b > largest ?	3	9	5	3	$9 > 3 \rightarrow$ yes
largest $\leftarrow$ b	3	9	5	9	b takes over
c > largest ?	3	9	5	9	$5 > 9 \rightarrow$ no
PRINT largest	3	9	5	9	output: 9

Learn to do this. When a program misbehaves, a trace table finds the bug faster than staring at the code does — and you can do it on paper, in the exam.

Three ways an algorithm goes wrong

1

Ambiguity

A step that two readers would carry out differently.

"Wait a bit"

2

A missing case

It works on your example and fails on the one you did not think of.

What if all three numbers are equal?

3

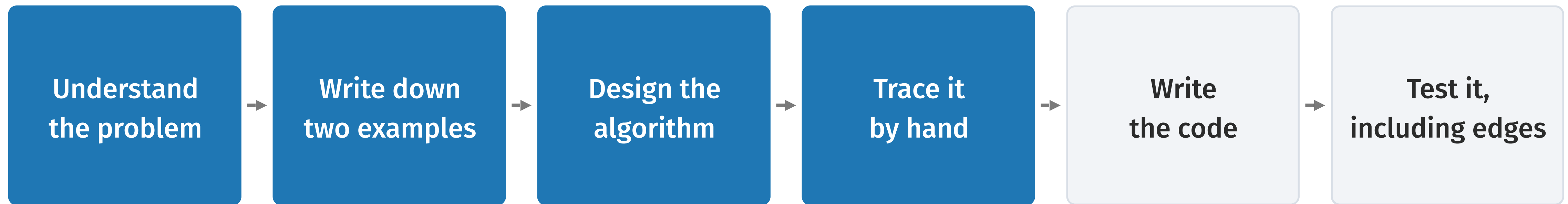
It never stops

A repetition whose condition never becomes false.

"Wait until the kettle boils" — with the kettle unplugged

A recipe for solving problems

Use it all semester



The four blue steps happen before you touch a keyboard.

Beginners skip straight to step five, get stuck, and conclude they are bad at programming. They are not — they are just solving a problem they have not understood yet.

Exercises — Block A

Sheet 01, part A · pen and paper · 40 minutes

- Write an algorithm a stranger could follow, for a task of your choice.
- Find the ambiguities in a given algorithm.
- Trace a given algorithm and report the output.
- Find the input for which a given algorithm produces the wrong answer.
- Turn a short flowchart back into pseudocode.

No laptops for this one. Work in pairs — swapping algorithms and trying to break each other's is the fastest way to learn what "unambiguous" means.



B

Your first Python program

Tooling · syntax · errors — on paper, or at a laptop

No laptop today? No problem

Three ways to follow this block — all of them work

Paper

Predict every result and write every fix by hand. Then we run them together on the projector.

Share

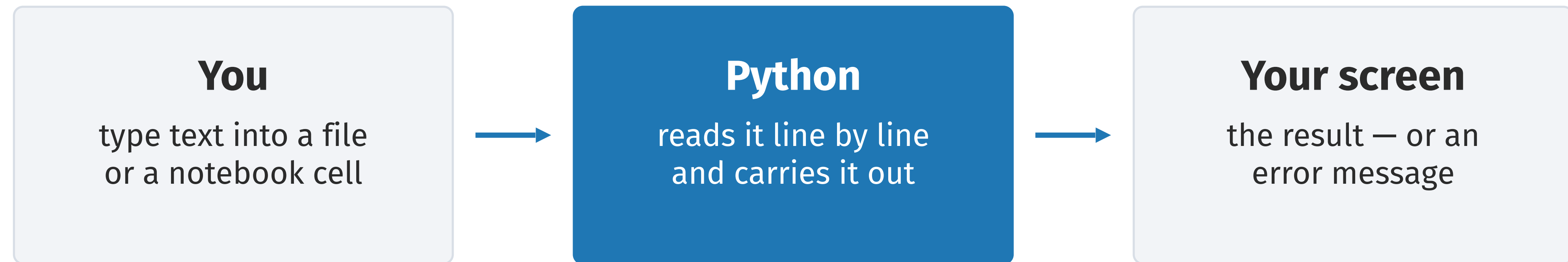
Pair up with someone who brought one. One types, one predicts — and swap.

Own laptop

Predict on paper first, then check. The prediction is the exercise; the machine is the marker.

Everything I demonstrate today happens on the screen at the front. Nothing in this block requires you to have installed anything — installation is homework, and I will help anyone who wants it after the lecture.

From your text to a result



Python never guesses what you meant.
It does exactly what you wrote — and if what you wrote makes no sense, it says so and stops.

Three ways to run Python

All three do the same thing

	REPL	Script (.py)	Notebook (.ipynb)
what it is	a prompt answering one line at a time	a file you run start to finish	cells you run in any order
good for	trying one thing out	real programs	learning and experiments
we use it for	quick checks	when programs outgrow a notebook	most lectures

```
$ python
>>> 2 + 2
4
>>> exit()
```

- The >>> prompt means Python is waiting for a line.
- It prints the value of whatever you type — a script does not do that, which surprises everyone exactly once.

Your first program

```
print("Hello, world")
```

Output

```
Hello, world
```

That is a complete, working program. Every programmer who ever lived started here — the tradition is older than most of the languages still in use.

Type it — or, with no laptop, write it out by hand. Do not copy and paste. The typos are part of the lesson.

Anatomy of one line

```
print("Hello, world")
```

print

the name of a function — a ready-made instruction that Python already knows

()

the parentheses say: run it now. Without them, nothing happens

"Hello, world"

the argument — what we hand to the function

" "

the quotes mark it as text. Without them Python looks for a variable by that name

Printing more than one thing

```
print("Hello", "world")
```

→ Hello world

```
print("2 + 2 =", 2 + 2)
```

→ 2 + 2 = 4

```
print()
```

→ an empty line

```
# this line does nothing at all
```

- Several arguments, separated by commas — Python puts a space between them.
- An argument can be a computation. Python works it out first, then prints the result.
- Anything after # is a comment: for humans, ignored by Python.

Arithmetic

Predict each answer, then we run them together

```
>>> 7 + 3
10
>>> 7 - 3
4
>>> 7 * 3
21
>>> 7 / 3
2.3333333333333335
```

```
>>> 7 // 3      # whole part
2
>>> 7 % 3       # remainder
1
>>> 7 ** 3      # power
343
>>> (7 + 3) * 2 # brackets first
20
```

Two of these will cause trouble later: / always gives a decimal, and that last decimal digit of 7 / 3 is not a typo. Lecture 02.

Errors are not failure

They are the most useful thing Python does

An error message is Python telling you where it got confused, and why.
Read it. It is almost always right.

```
Traceback (most recent call last):  
  File "hello.py", line 3, in <module>  
    print(greting)  
NameError: name 'greting' is not defined
```

- Read the LAST line first — it names the problem.
- Then the line number — it says where.
- The middle is the path Python took to get there. Ignore it for now.

Nobody writes code without errors. Experienced programmers just read them faster.

Four errors you will meet today

1 — `SyntaxError`

```
print("Hello, world)
```

`SyntaxError: unterminated string literal (detected at line 1)`

- The closing quote is missing, so Python never finds where the text ends.
- `SyntaxError` means Python could not even read your program — nothing ran at all.
- Look at the reported line, and at the line above it:
→ an unclosed bracket or quote is often reported one line late.

Four errors you will meet today

2 — *IndentationError*

```
print("first")
    print("second")
```

IndentationError: unexpected indent

- In Python, leading spaces are not decoration — they are part of the grammar.
- A line that starts further right means "this belongs inside the thing above".
- Nothing is above it here, so the indent makes no sense.
- From lecture 03 on, indentation is how you say what is inside an if or a loop.

Four errors you will meet today

3 — `NameError`

```
message = "Hello"  
print(mesage)
```

`NameError: name 'mesage' is not defined. [Did you mean: 'message'?]`

- Python knows the names you gave it, and nothing else.
- One letter wrong is a different name — and it has no idea what you meant.
- Python suggests the name you probably meant — but only in a script (.py) or the REPL. Jupyter formats its errors itself and leaves the suggestion out. Only from Python ≥ 3.10
- Also happens when you use a name before creating it.

Four errors you will meet today

```
print("Result: " + 42)
```

`TypeError: can only concatenate str (not "int") to str`

- + means "join" for text and "add" for numbers. Python will not mix the two.
- Fix one — a comma instead of a plus:

```
print("Result: ", 42)
```

```
print("Result: " + str(42))
```

Fix two converts the number to text first. Both are correct; we will see when each is preferable.

How to get unstuck

In this order

1 Read the last line of the error

It names the problem in four words.

2 Go to the line number

And look at the line above it too.

3 Print things

Put `print()` before the failing line and look at what the values actually are.

4 Change one thing

Then run again. Never change three things at once.

5 Ask

A classmate, then me.
Bring the error message, not "it does not work".

Using an AI assistant is allowed and will be useful. Using it before you have read the error yourself will cost you the exam.

Exercises — Block B

Sheet 01, part B · 40 minutes · paper first, always

Everyone, on paper

- Predict the output of eight expressions.
- Name the error in six broken lines — and write the fix.
- Write out a small receipt program by hand.
- Work out a change calculation with the right operators.

If you have a laptop, then check

- Run what you predicted and compare.
- Where you were wrong, work out why before moving on.
- Share the screen with a neighbour who has no machine.

Then we run them together. You dictate, I type — including the versions you think are wrong.

Today in one slide

A computer does five things

store, fetch, compute, compare, repeat — very fast, and with no judgement whatsoever.

Everything is bits

the meaning comes from the type, not from the bits.

An algorithm is the idea

finite, unambiguous, effective, with defined input and output. The language comes after.

Think before you type

understand, example, algorithm, trace — then code.

Python does what you wrote

not what you meant.

Errors are information

read the last line, go to the line number, change one thing.

Setup before lecture 02

Pick one — all four work for this course

A • Python + VS Code

Install Python, then VS Code with the Python and Jupyter extensions.

Small download, nothing to license. Recommended.

python.org
code.visualstudio.com

B • Google Colab

Nothing to install.
Notebooks run in the browser.

Needs a Google account and a connection.

colab.research.google.com

C • Miniforge

conda, preconfigured to the community conda-forge channel.

conda create -n prog26
python=3.12 jupyter

github.com/conda-forge/miniforge

D • Anaconda

One installer: Python, Jupyter, and the scientific libraries.

Anaconda is a several-GB download.

anaconda.com/download

```
python --version          # macOS: python3 --version    ·    Windows: py --version
python -c "print('ready')"
```

Windows: tick "Add python.exe to PATH" in the installer, or typing python opens the Microsoft Store.

- *macOS: the command is python3.*
- *Stuck? Stay ten minutes after the lecture and we will fix it together.*

Before next time

- Install Python at home — the previous slide has the four options. Ten minutes, once.
- Finish sheet 01 if you did not get through it. Solutions go online after this lecture.
- Read chapter 1.1 of Sedgewick, Wayne & Dondero if you want a second voice on today.
- Bring a laptop from lecture 02 if you possibly can. If you cannot, come and tell me — we will sort something out, and the paper track keeps running.

Next: Lecture 02 — values, variables, data types and expressions.